# LAB SESSION 1: Source Coding

## Background and explanations:

***Objectives*:** The main aim of this session is to learn:

1) How to start Matlab?
2) Understand digital communication systems.
3) Implement entropy calculation for information content.
4) Explore data compression using Huffman coding.

## *Starting Matlab (recap):*

To start Matlab, double click on the Matlab icon on the desktop. The main Matlab window with the command line prompt ">>" should appear.

Matlab is an interpreted language. This means that the instructions you give to it are processed and interpreted by the computer directly, as opposed to for instance C; in C, when you write a program, you first need to compile it before it can be executed by the machine. This means that at any moment, all the variables are in the memory of the computer (equivalent to debug mode of C/C++) and can be readily accessed. This is a tremendous help to quickly debug programs.

There is two ways to give instructions to Matlab:

1- On the command line, after the prompt ">>": if you type an instruction, Matlab executes it and gives you the result.

2- Through a script or a function, whose code is written in a text file with the *.m* extension. Scripts contain lists of instructions, just as could be written on the command line. To execute a script, just type the name of the .m file (without the .m extension) on the command line[1]. When a script is launched, all the instructions it

[1] Matlab needs to see this file. For this, it should either be physically in your current directory (type ">>pwd" to find out where you are), or the path to its location should be known by matlab (type ">>path" to see the full path; type ">>help path" if you need to change it). Go to the File menu and Set Path sub-menu to change the path.

contains are executed successively. Functions are modular scripts that can take in arguments. They won't be detailed here, but you can look at the on-line help to find out about them.

## *Understanding Digital Signals*

### *Concept*

- Digital communication transmits data as bits (0s and 1s).
- Noise affects transmission and data integrity.
- Compression reduces transmission size while maintaining information.

## *Task 1: Generate and Plot a Digital Signal*

Generate a random binary sequence and plot the digital signal (you may save following code in .m file).

### MATLAB Code

```
clc; clear; close all;

% Generate a random binary sequence
N = 100; % Number of bits
binary_seq = randi([0, 1], 1, N);

% Time vector
t = 0:N-1;

% Plot digital signal
figure;
stairs(t, binary_seq, 'LineWidth', 2);
ylim([-0.5, 1.5]);
xlabel('Time');
ylabel('Signal');
title('Digital Signal Representation');
grid on;
```

### Expected Output

A staircase plot representing a random digital signal.

## Task 2: Entropy Calculation

**Concept**

- **Entropy** measures the uncertainty (information content) of a signal.
- A balanced dataset (50% 0s, 50% 1s) has **maximum entropy**.
- A dataset with mostly 0s or mostly 1s has **low entropy**.

## Compute Entropy of a Binary Source

Compute entropy based on probability.

**MATLAB Code**

```
% Generate a random binary sequence
N = 1000; % Number of bits
binary_seq = randi([0, 1], 1, N);

% Compute probability of 0s and 1s
p0 = sum(binary_seq == 0) / N;
p1 = sum(binary_seq == 1) / N;

% Print probabilities of 0 and 1

fprintf('Probability of 0 : %.4f bits\n', p0);
fprintf('Probability of 1 : %.4f bits\n', p1);

% Shannon entropy formula
entropy_value = - (p0 * log2(p0) + p1 * log2(p1));

% Display results
fprintf('Entropy of the digital signal: %.4f bits\n', entropy_value);
```

### Expected Output

Entropy value between **0 and 1 bit per symbol**.

## Task 3: Huffman Coding for Data Compression Concept

- **Huffman coding** reduces the size of data by assigning shorter codes to frequent symbols.
- The entropy gives the lower bound on the average code length.

**Task: Compress a Text Message**

You will implement Huffman coding to compress a given message.

**MATLAB Code**

1) Initialise matlab workspace.

```
clc;
clear;
close all;
```

2) Ask user to input a text string, count frequencies and calculate and print probabilities.

```
% -------------------------
% MAIN SCRIPT
% -------------------------

% Input sentence
sentence = input('Enter a sentence: ', 's');

% Count character frequencies
symbols = unique(sentence); % Get unique characters
freqs = arrayfun(@(c) sum(sentence == c), symbols); % Count occurrences
probs = freqs / sum(freqs); % Calculate probabilities

% Convert symbols to a cell array (Huffman requires this format)
symbols_cell = num2cell(symbols);

% Display probabilities
disp('Character Probabilities:');
for i = 1:length(symbols)
    fprintf('%c: %.4f\n', symbols(i), probs(i));
end
```

3) Calculate and display Entropy

```
% Calculate and display entropy
entropy_value = -sum(probs .* log2(probs));
disp(['Entropy: ', num2str(entropy_value)]);
```

4) Huffman Encoding and display the coded sentence, code words

```
% Huffman encoding
dict = huffmandict(symbols_cell, probs);

% Encode the sentence
coded_sentence = huffmanenco(num2cell(sentence), dict);

disp(['Coded Sentence: ', num2str(coded_sentence)]);

% Display Huffman code words
disp('Huffman Code Words:');
for i = 1:length(dict)
    fprintf('%c: %s\n', dict{i,1}, num2str(dict{i,2}));
end
```

5) Calculate compression Ratio

```
% Compute Original and Compressed Data Size
original_bits = length(sentence) * 8; % Assuming ASCII encoding (8 bits per
character)
compressed_bits = length(coded_sentence); % Total bits in Huffman encoded
message

% Compute Compression Ratio
compression_ratio = original_bits / compressed_bits;
disp(['Compression Ratio: ', num2str(compression_ratio)]);
```

6) Calculate average code length

```
% Calculate average code length correctly
code_lengths = cellfun(@length, dict(:,2));
avg_code_length = sum(probs(:) .* code_lengths(:));
```

7) Calculate Compression efficiency

```
% Calculate compression efficiency
if avg_code_length > 0
    compression_efficiency = entropy_value / avg_code_length;
else
    compression_efficiency = 0;
end

disp(['Average Code Length: ', num2str(avg_code_length)]);
disp(['Compression Efficiency: ', num2str(compression_efficiency)]);
```

8) Huffman decoding to retrieve the original text

```
% Huffman decoding
decoded_sentence = char(cell2mat(huffmandeco(coded_sentence, dict)));
disp(['Decoded Text: ', decoded_sentence]);
```

9) If you would like to see the Huffman tree, then insert the following code at the end

```
% -------------------------
% BUILDING & DISPLAYING HUFFMAN TREE
% -------------------------

global node_id edges labels;
node_id = 1; % Reset node counter
```

```
    % Build Huffman tree from symbols and probabilities
    root_node = buildTree(symbols_cell, probs);

    % Step 2: Create and plot the graph
    G = digraph(edges(:,1), edges(:,2)); % Create directed graph

    figure;
    plot(G, 'Layout', 'layered', 'NodeLabel', labels);
    title('Huffman Tree');

% -------------------------
% BUILDING & DISPLAYING HUFFMAN TREE
% -------------------------

global node_id edges labels;
node_id = 1; % Reset node counter

% Build Huffman tree from symbols and probabilities
root_node = buildTree(symbols_cell, probs);

% Step 2: Create and plot the graph
G = digraph(edges(:,1), edges(:,2)); % Create directed graph

figure;
plot(G, 'Layout', 'layered', 'NodeLabel', labels);
title('Huffman Tree');

% -------------------------
% FUNCTION TO BUILD HUFFMAN TREE
% -------------------------
function root = buildTree(symbols, probs)
    global node_id edges labels;
    node_id = 1;  % Reset node counter
    edges = [];   % Stores parent-child edges
    labels = {};  % Stores labels for visualization

    % Create a priority queue (cell array of nodes)
    nodes = cell(length(symbols), 1);
    for i = 1:length(symbols)
        nodes{i} = struct('symbol', symbols{i}, 'prob', probs(i), 'left', [],
'right', [], 'id', node_id);
        labels{node_id} = symbols{i}; % Label each leaf with its symbol
        node_id = node_id + 1;
    end

    % Iteratively merge nodes until only one remains
    while length(nodes) > 1
        % Sort nodes by probability (ascending order)
        [~, idx] = sort(cellfun(@(x) x.prob, nodes));
        nodes = nodes(idx);

        % Select two nodes with lowest probabilities
        left = nodes{1};
        right = nodes{2};

        % Create a new parent node
        parent = struct('symbol', [left.symbol, right.symbol], 'prob', left.prob +
right.prob, ...
```

```
                        'left', left, 'right', right, 'id', node_id);
        labels{parent.id} = strcat(left.symbol, '+', right.symbol);

        % Add edges to track parent-child relationships
        edges = [edges; parent.id, left.id; parent.id, right.id];

        % Remove merged nodes and insert the new parent
        nodes = [{parent}; nodes(3:end)];
        node_id = node_id + 1;
    end

    % The last remaining node is the root
    root = nodes{1}.id;
end
```

**Expected Output**

**Save the above code in a .m file and observe the results for various text.**

# Questions

1. **Digital Signals:**
   - What are the advantages of digital over analog signals?
   - How does noise affect digital communication?
   - Explain how transmission bandwidth affects digital communication.
2. **Entropy Calculation:**
   - What does entropy represent in communication?
   - How does the entropy value change if the dataset has more 1s than 0s (Task 2)?
3. **Huffman Coding:**
   - Why does Huffman coding provide better compression?
   - How does Huffman coding relates to entropy?
4. A message X has five symbols x1 , x2, x3, x4, and x5 with P(x1 ) = 0.4, P(x2) = 0.2, P(x3) = 0.1, P(x4) = 0.2, and P(x5 ) = 0.1.
   a) Construct the Huffman tree for X, and calculate Huffman code.
   b) Calculate entropy and the efficiency of the code (Avg code length/entropy).

First, solve question 4 manually on paper, then by using an appropriate message, compare your results with the output of the computer program above.

5. A Discrete Memoryless Source (DMS) X has four symbols $x_1$ $x_2$, $x_3$, $x_4$ with probabilities (solve question manually)

   $P(x_1) = 0.4$, $P(x_2) = 0.3$, $P(x_3) = 0.2$, $P(x_4) = 0.1$.

(i) Calculate H(X).

(ii) Find the amount of information contained in the messages $x_1$ $x_2$ $x_1$ $x_3$ and $x_4$ $x_3$ $x_3$ $x_2$, and compare with the H(X) obtained in part (i).